Programação 2

Aula 5 - Orientação a Objetos: classe e objeto, construtores

LUCAS SILVA FIGUEIREDO
UNIVERSIDADE FEDERAL RURAL DE PERNAMBUCO

Recapitulando

Métodos

Estruturas de declaração dos métodos

```
nivelDeAcesso tipoDeRetorno nomeDoMetodo(tipo parametro1, ...){
    // passos da execução do método
    return variavelDeRetorno; // não é usado para métodos void
}
```

Em que:

```
nivelDeAcesso = public ou private
tipoDeRetorno = tipo de variável (ex.: int, String) ou void (sem retorno)
return = instrução indicando o retorno de uma variável
parametrosN = variáveis locais acessíveis no escopo do método
```

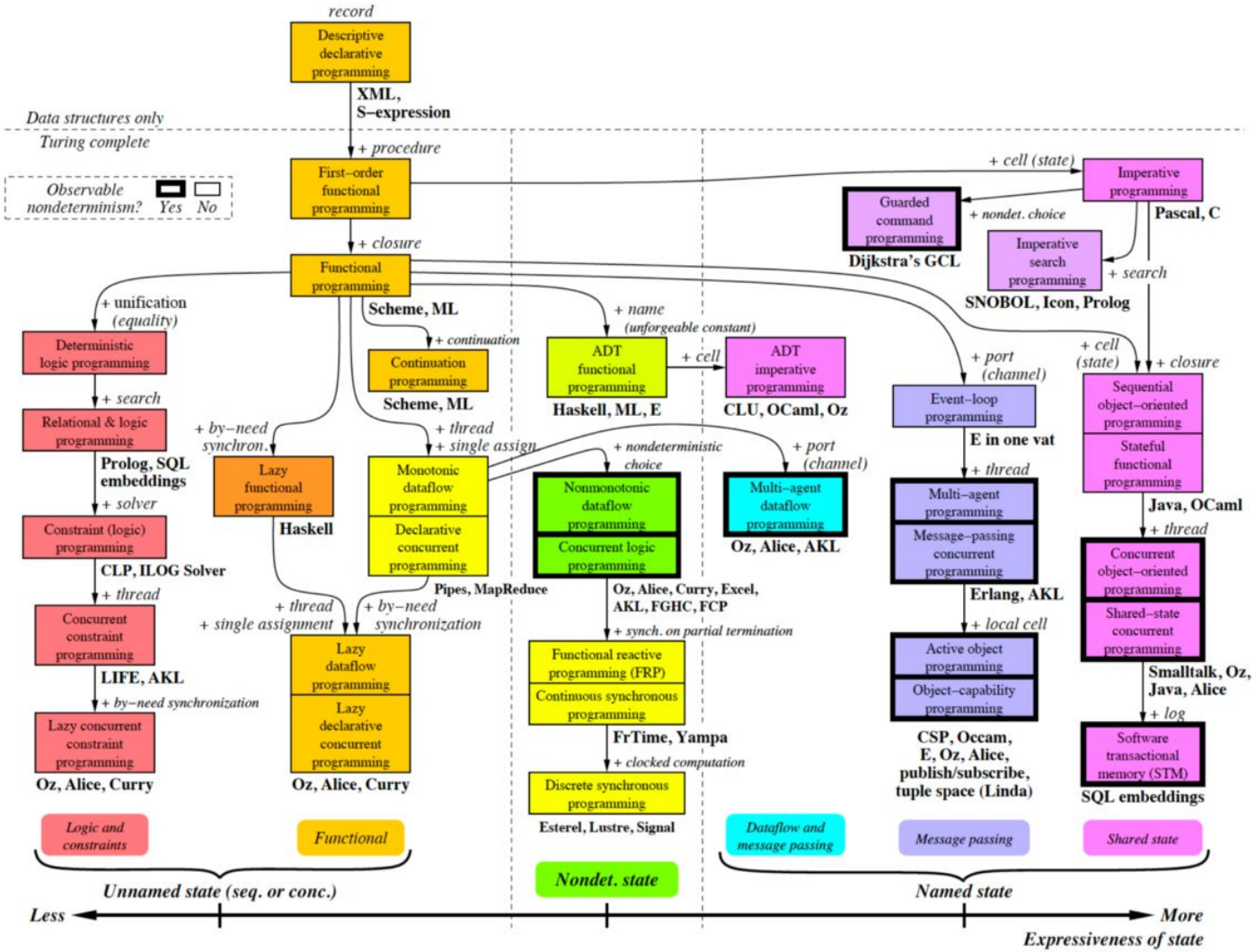
Métodos

Estruturas de chamadas de métodos

```
nomeDoObjeto.nomeDoMetodo(parametro1, parametro2, ...)
```

Paradigmas de Linguagens de Programação

LUCAS SILVA FIGUEIREDO
UNIVERSIDADE FEDERAL RURAL DE PERNAMBUCO



[Classification of the principal programming paradigms (ucl.ac.be)](#)

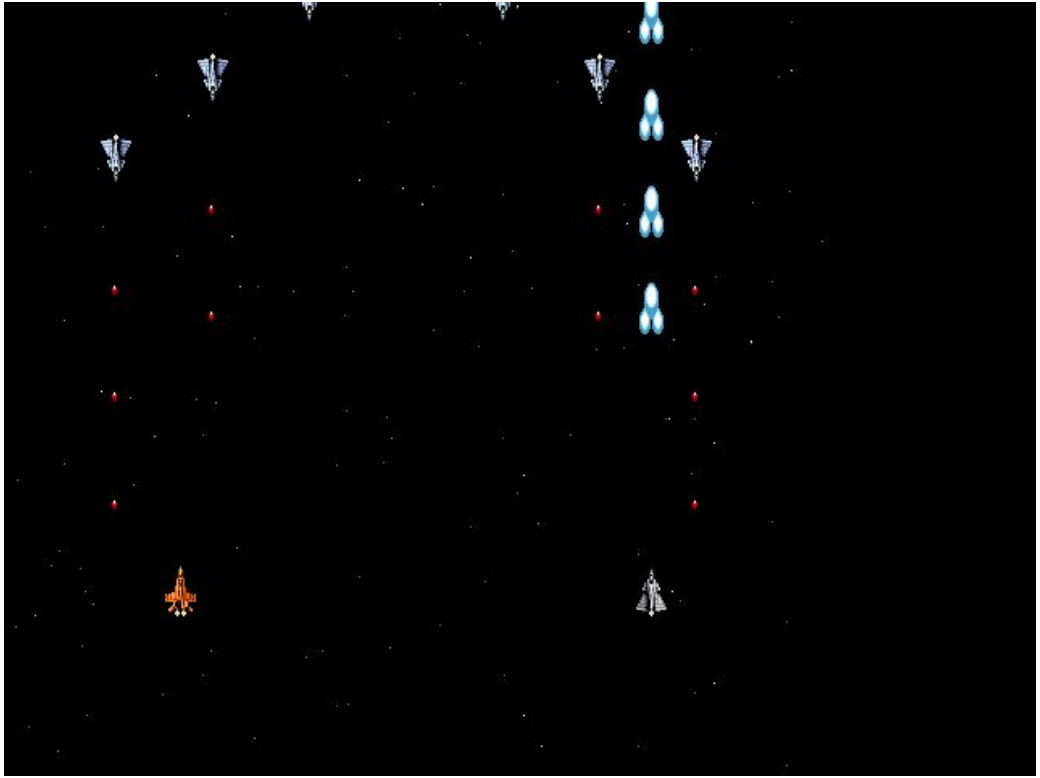
[A Brief Survey of "Programming Paradigms" by Jing Chen | Medium](#)

LUCAS SILVA FIGUEIREDO

UNIVERSIDADE FEDERAL RURAL DE PERNAMBUCO

Paradigma Procedural (ex.: C)

C



801 lines (724 sloc) | 22.6 KB

```c
#include <allegro.h>
#include <stdlib.h>

#include "constantes.h"

#define MAX_X         640
#define MAX_Y         480
#define V_MAX_X       0
#define V_MAX_Y       0
#define COLOR_BITS    16

#define DELAY_GIRO    15

        typedef struct {
                int px;
                int py;
                int vx;
                int vy;
                int pintar;
        } Bala;

        typedef struct {
                Bala **balas1;
                BITMAP *bitbal1;
                Bala **balas2;
                BITMAP *bitbal2;
                Bala **balas3;
                BITMAP *bitbal3;
```

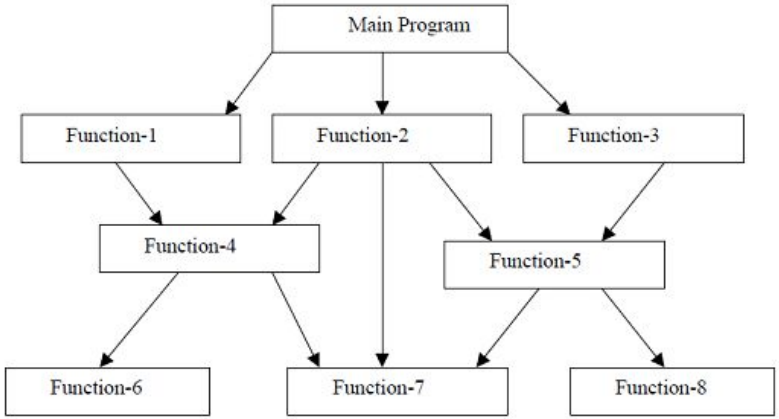
[lsfcin/aero-wings: Game coded in 2007 (in between semesters) using Allegro, inspired in AeroFighters and SonicWings! (github.com)]

C

```c
int main() {
 allegro_init();
 install_keyboard();
 set_color_depth(COLOR_BITS);
 if (set_gfx_mode(GFX_AUTODETECT, MAX_X, MAX_Y, V_MAX_X, V_MAX_Y) < 0) {
        //printf("Erro ao carregar o modo de vídeo!");
        return 0;
 }

 //fundo
 BITMAP *buffer = create_bitmap(MAX_X, MAX_Y);
 Scroll *scroll1 = carregarScroll("rsc/fundo2.bmp", NULL);
 Scroll *scroll2 = carregarScroll("rsc/fundo2.bmp", NULL);
 setScroll(scroll1, -600, 1);
 setScroll(scroll2, -1800, 1);

 Scroll *scroll3 = carregarScroll("rsc/fundo3.bmp", NULL);
 Scroll *scroll4 = carregarScroll("rsc/fundo3.bmp", NULL);
 setScroll(scroll3, -600, 1);
 setScroll(scroll4, -1800, 1);

 //tempo e posicao
 int tempo = 0;
 int loopt = 0;

 //tela
 Tela * tela = carregarTela();
```

C

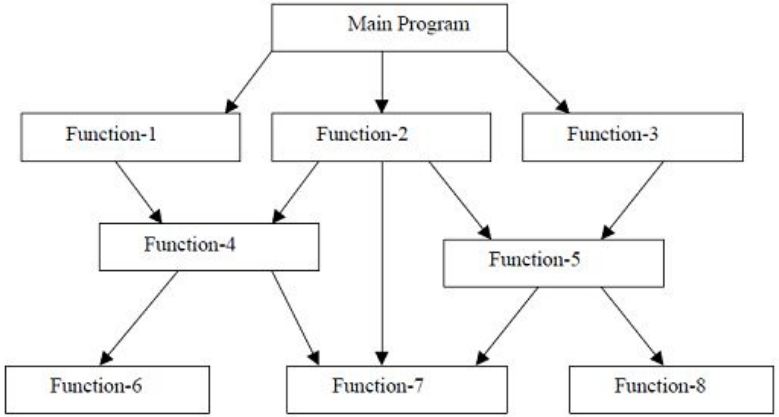
```c
int main() {
 allegro_init();
 install_keyboard();
 set_color_depth(COLOR_BITS);
 if (set_gfx_mode(GFX_AUTODETECT, MAX_X, MAX_Y, V_MAX_X, V_MAX_Y) < 0) {
        //printf("Erro ao carregar o modo de vídeo!");
        return 0;
 }

 //fundo
 BITMAP *buffer = create_bitmap(MAX_X, MAX_Y);
 Scroll *scroll1 = carregarScroll("rsc/fundo2.bmp", NULL);
 Scroll *scroll2 = carregarScroll("rsc
 setScroll(scroll1, -600, 1);
 setScroll(scroll2, -1800, 1);

 Scroll *scroll3 = carregarScroll("rsc
 Scroll *scroll4 = carregarScroll("rsc
 setScroll(scroll3, -600, 1);
 setScroll(scroll4, -1800, 1);

 //tempo e posicao
 int tempo = 0;
 int loopt = 0;

 //tela
 Tela * tela = carregarTela();
```



Structure of procedural oriented programs

[Programming in C++ - Procedure Oriented Programming Vs OOP(Object Oriented Programming) - EXAMRADAR]

Procedural
(ex.: C)



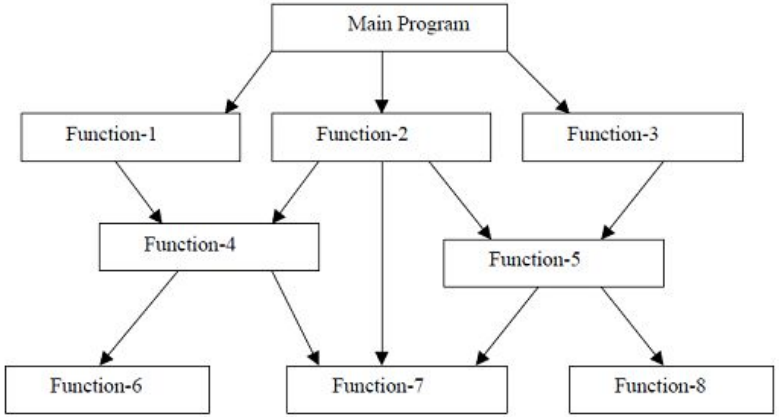
[Procedure Oriented Programming Vs OOP(Object Oriented Programming) - EXAMRADAR]

Conceito de Hoje

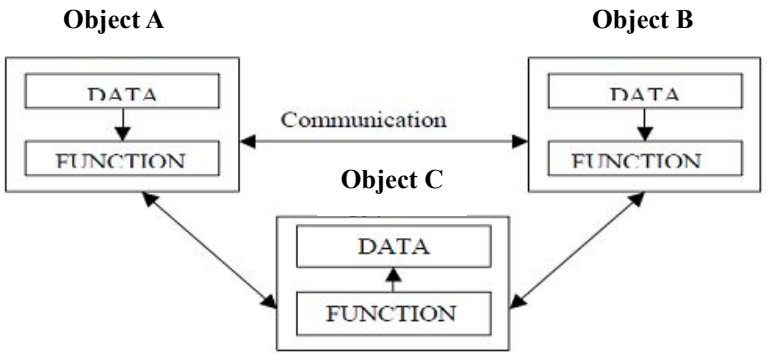
Paradigma Orientado a Objetos (ex.: Java)

LUCAS SILVA FIGUEIREDO
UNIVERSIDADE FEDERAL RURAL DE PERNAMBUCO

Procedural
(ex.: C)

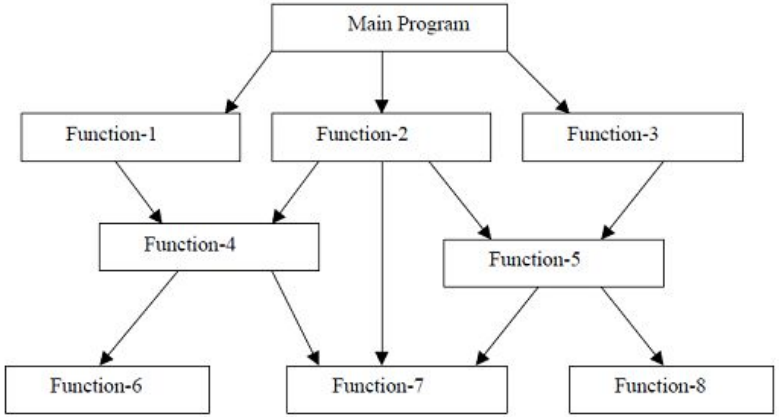


Orientada a Objetos
(ex.: Java)



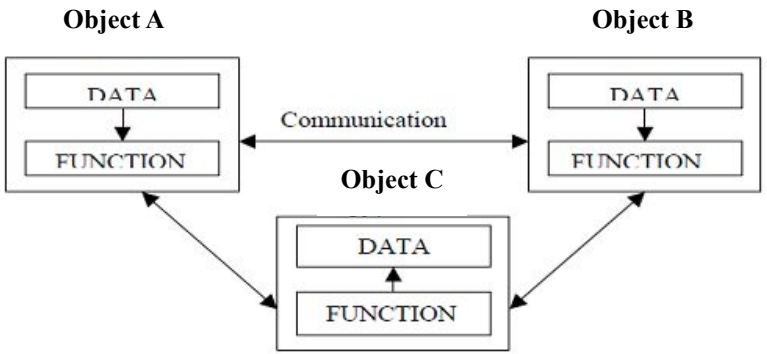
[Procedure Oriented Programming Vs OOP(Object Oriented Programming) - EXAMRADAR]

Procedural
(ex.: C)



- Emphasis is on doing things (algorithms).
- Large programs are divided into smaller programs known as functions.
- Most of the functions share global data.
- Data move openly around the system from function to function.
- Functions transform data from one form to another.
- Employs top-down approach in program design.

Orientada a Objetos
(ex.: Java)



- Emphasis is on data rather than procedure.
- Programs are divided into what are known as objects.
- Data structures are designed such that they characterize the objects.
- Functions that operate on the data of an object are ties together in the data structure.
- Data is hidden and cannot be accessed by external function.
- Objects may communicate with each other through function.
- New data and functions can be easily added whenever necessary.
- Follows bottom up approach in program design.

[Procedure Oriented Programming Vs OOP(Object Oriented Programming) - EXAMRADAR]

Objetos

Objetos

São entidades

- Idealmente com responsabilidades específicas

Guardam dados

Executam métodos

Objetos: exemplos de objetos no nosso código

```
Scanner escaneador = new Scanner(System.in);
```

```
escolha = escaneador.nextLine();

// Em orientação a objetos o próprio objeto avalia seu tamanho, por exemplo
int tamanho = escolha.length();

// Se fosse procedural, seria algo do tipo
int tamanho = funcaoAvaliaTamanho(escolha);
```

Classes

Classe e Objeto

Classe

Objetos produzidos

Parâmetros do solicitante, ex.:
- Cor do carro
- Motor (1.0, 1,6)
- Sedan, hatch

Classe

Imagine a classe como uma fábrica de objetos

É quem “produz” ou “**constrói**” um objeto

Uma fábrica de **pães** produz pães

Uma fábrica de **chocolate** produz chocolate

Uma fábrica de **carros** produz carros

E assim como uma fábrica, a classe pode receber parâmetros para produzir um objeto específico

O tipo de farinha que se usa no pão pode ser um parâmetro

As quantias de açúcar e leite do chocolate...

A cor do carro pode ser um parâmetro

Java (Orientada a Objetos)

Objeto

Entidade, caso real, existência, **instanciação** da classe

Classe

Definição

Quais são as características (atributos, propriedades)?

Quais são as ações (métodos)?

Classe e Objeto sobre a entidade “Cachorro”

Classe

Dados/Atributos
- Raça
- Tamanho
- Cor do Pelo
- Cor dos Olhos
- Sexo
- Tamanho do Rabo
- Tamanho do Pelo

Ações/Métodos
- Latir
- Morder
- Farejar
- Bagunçar
- Rasgar Sacolas
- Comer
- Correr

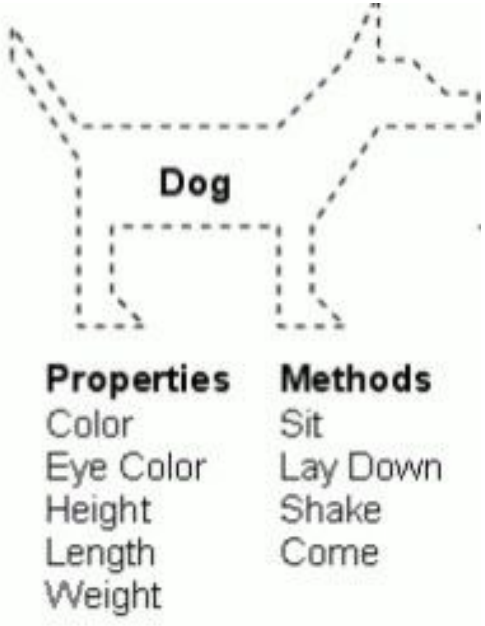
Objeto

Dados/Atributos
- Raça: ChowChow
- Tamanho: 1m
- Cor do Pelo: Caramelo
- Cor dos Olhos: Preto
- Sexo: Masculino
- Tamanho do Rabo: 30cm
- Tamanho do Pelo: Longo

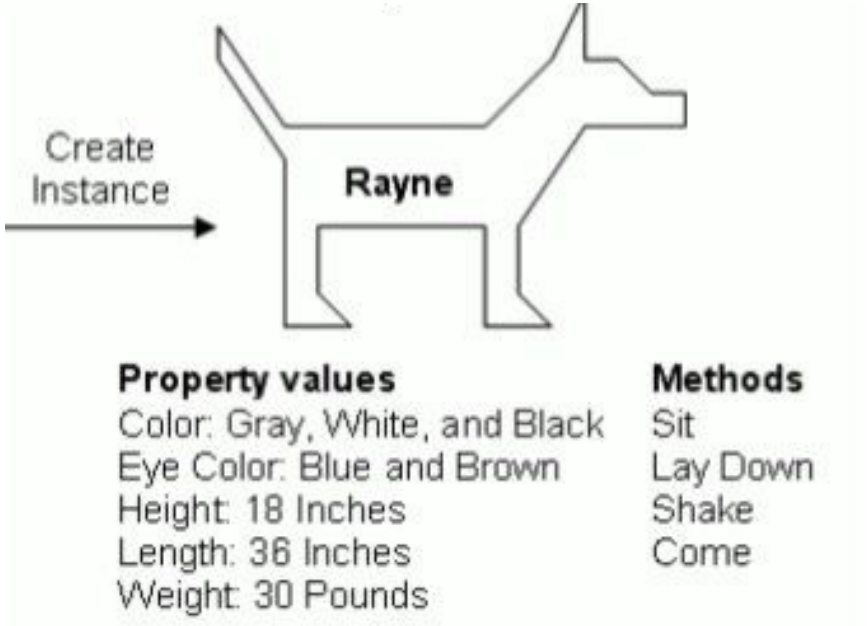
Ações/Métodos
- Latir
- Morder
- Farejar
- Bagunçar
- Rasgar Sacolas
- Comer
- Correr

Classe e Objeto sobre a entidade “Cachorro”

Classe



Objeto



[A touch of class: Understanding OOP]

Unified Modeling Language - **UML**

Forma visual (diagramas) para representar classes e as interações entre elas

Classe Cachorro
- Idade - Peso - Altura - Nome - Raça - Personalidade - Cor/Pelagem - ...
+ Comer() + Dormir() + Bagunçar() + Correr()

Criação, Construtores e **new**

Classes: Criação

Como criamos a classe cachorro?

Classe Cachorro

```
public class Cachorro {
    String raca;
    float tamanhoDoRabo;
    String corDosOlhos;
    String corDoPelo;
}
```

Classes: Criação

Qual cachorro esta classe representa?

Todos os cachorros

O que é o termo "**class**"?

Denota que é uma classe ou "fábrica"

O que é o termo "**Cachorro**"?

o nome da classe e também o que ela cria

O que significa as **chaves** "{ }" ?

o escopo da classe, tudo que ela sabe fazer

O que é o termo **public**?

"Todo mundo pode usar"

```
1
2 public class Cachorro {
3
4 }
5
```

Classes: Criação: Atributos

Como criamos os atributos (*variáveis de instância*) da classe cachorro?

Quais são os tipos desses atributos?

Como incluir a cor em RGB do cachorro?

Classe Cachorro
- Idade - Peso - Altura - Nome - Raça - Personalidade - Cor/Pelagem - ...

Classes: Criação: Atributos: Composição

Objetos de uma classe podem ter **atributos** (*variáveis da instância*) que não são tipos primitivos, mas sim criados a partir de **outras classes**

```
class Cachorro {
    Image foto;
}
```

Classes: Criação: Atributos: Composição

Objetos de uma classe podem ter **atributos** (*variáveis da instância*) que não são tipos primitivos, mas sim criados a partir de **outras classes**

```java
public class Dog {
    private int age; // Idade
    private float weight; // Peso
    private float height; // Altura
    private String name; // Nome
    private String race; // Raça
    private String personality; // Personalidade
    private Color furColor; // Cor do Pelo

}
```

```java
public class Color {
    private int r;
    private int g;
    private int b;
}
```

Objetos: Criação

Como criamos um objeto do tipo cachorro?

Cachorro X?

Objetos: Criação: Atribuição

Como atribuímos os valores de cada um dos atributos (*variáveis da instância*) de um objeto do tipo cachorro?

Cachorro X
- Idade - Peso - Altura - Nome - Raça - Personalidade - Cor/Pelagem - ...
+ Comer() + Dormir() + Bagunçar() + Correr()

Objetos: Criação: Construtores

Alternativamente, podemos criar um **método construtor** que sempre será chamado toda vez que usarmos o comando **new** para a nossa classe.

No construtor podemos indicar todos os parâmetros que consideramos necessários para inicializar o nosso objeto.

Como criar um construtor para a nossa classe?

Objetos: Criação: Construtores

No código...

```java
public class Cor {
    int r;
    int g;
    int b;

    Cor(int r, int g, int b) {
        this.r = r;
        this.g = b;
        this.b = b;
    }
}
```

```java
public class Cachorro {
    String raca;
    float peso;
    float tamanho;
    float tamanhoDoPelo;
    Cor corDoPelo;

    Cachorro(String raca, float peso, float tamanho, float tamanhoDoPelo, Cor corDoPelo)
    {
        this.raca = raca;
        this.peso = peso;
        this.tamanho = tamanho;
        this.tamanhoDoPelo = tamanhoDoPelo;
        this.corDoPelo = corDoPelo;
    }
```

Exemplo de Classe

Classe Account (Conta) em Java

O que é **atributo**?

O que é **método**?

```java
public class Account
{
    private String name; // variável de instância

    // método para definir o nome no objeto
    public void setName(String name)
    {
        this.name = name; // armazena o nome
    }

    // método para recuperar o nome do objeto
    public String getName()
    {
        return name; // retorna valor do nome para o chamador
    }
} // fim da classe Account
```

Classe AccountTest em Java

O que é o **main**?

O que é o comando **new**?

```java
public class AccountTest
{
   public static void main(String[] args)
   {
      // cria um objeto Scanner para obter entrada a partir da janela de comando
      Scanner input = new Scanner(System.in);

      // cria um objeto Account e o atribui a myAccount
      Account myAccount = new Account();

      // exibe o valor inicial do nome (null)
      System.out.printf("Initial name is: %s%n%n", myAccount.getName());

      // solicita e lê o nome
      System.out.println("Please enter the name:");
      String theName = input.nextLine(); // lê uma linha de texto
      myAccount.setName(theName); // insere theName em myAccount
      System.out.println(); // gera saída de uma linha em branco

      // exibe o nome armazenado no objeto myAccount
      System.out.printf("Name in object myAccount is:%n%s%n",
         myAccount.getName());
   }
} // fim da classe AccountTest
```

Código Produzido em Sala

```java
public class App {

    Run | Debug
    public static void main(String[] args) throws Exception
    {
        Cor caramelo = new Cor(r: 255, g: 232, b: 152);
        Cor bege = new Cor(r: 255, g: 242, b: 182);
        Cor preto = new Cor(r: 0, g: 0, b: 0);
        Cachorro chowChow = new Cachorro(
            raca: "Chow Chow",
            tamanhoDoRabo: 0.30f,
            caramelo,
            preto);

        Cachorro goldenRetriever = new Cachorro(
            raca: "Golden Retriever",
            tamanhoDoRabo: 0.50f,
            rCorDosOlhos: 12,
            gCorDosOlhos: 12,
            bCorDosOlhos: 12,
            rCorDoPelo: 12,
            gCorDoPelo: 12,
            bCorDoPelo: 12);

        System.out.println();
        System.out.println(chowChow.raca);
        System.out.println();
    }
}
```

```java
public class Cachorro {
    String raca;
    float tamanhoDoRabo;
    Cor corDosOlhos;
    Cor corDoPelo;

    public Cachorro(String raca,
                    float tamanhoDoRabo,
                    Cor corDosOlhos,
                    Cor corDoPelo)
    {
        this.raca = raca;
        this.tamanhoDoRabo = tamanhoDoRabo;
        this.corDosOlhos = corDosOlhos;
        this.corDoPelo = corDoPelo;
    }

    public Cachorro(String raca,
                    float tamanhoDoRabo,
                    int rCorDosOlhos,
                    int gCorDosOlhos,
                    int bCorDosOlhos,
                    int rCorDoPelo,
                    int gCorDoPelo,
                    int bCorDoPelo)
    {
        this(raca,
             tamanhoDoRabo,
             new Cor(rCorDosOlhos, gCorDosOlhos, bCorDosOlhos),
             new Cor(rCorDoPelo, gCorDoPelo, bCorDoPelo));
    }
}
```

```java
public class Cor {
    int r;
    int g;
    int b;

    public Cor(int r, int g, int b)
    {
        this.r = r;
        this.g = g;
        this.b = b;
    }
}
```

Prática

Mini-Projeto 2

Criar uma classe representando um **personagem** da história interativa.

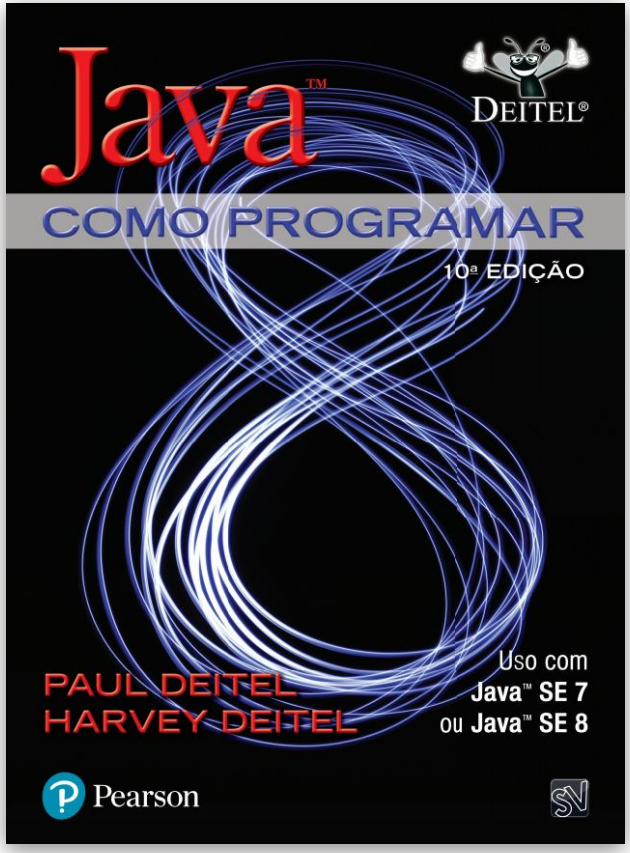
Cada **personagem** deve ter pelo menos os atributos: **String** nome e **int** energia

Criar pelo menos **dois objetos** (ou seja, dois personagens) usando o construtor dessa classe, representando instâncias da classe, com atributos definidos.

Cada capítulo deve ter como foco o desenvolvimento da história de um dos personagens. Assim, ao mostrar um capítulo na tela algo acontecerá com o personagem central neste capítulo. **A energia deste personagem deve mudar ao mostrar o capítulo**, nunca passando do máximo e sempre que chegar a zero você deve lançar uma mensagem explicando que o personagem "ficou esgotado", "pereceu" ou de outra forma que queira. Busque sempre contextualizar a perda ou o ganho de energia.

> // Esta seria a mensagem original do capítulo e em negrito abaixo a mensagem da perda de energia
> *Ex.: Ao pular o abismo entre dois prédios de forma exitosa Neo cai de cara no asfalto.*
> *Impressionantemente está vivo, mas mesmo a Matrix sendo uma ilusão*
> *causa danos no seu corpo físico, mais especificamente na sua mente.*
> ***Neo perde 15 pontos de energia.***

Referência

Livro:

Java - Como Programar (10a Edição)
[Capítulo 3]